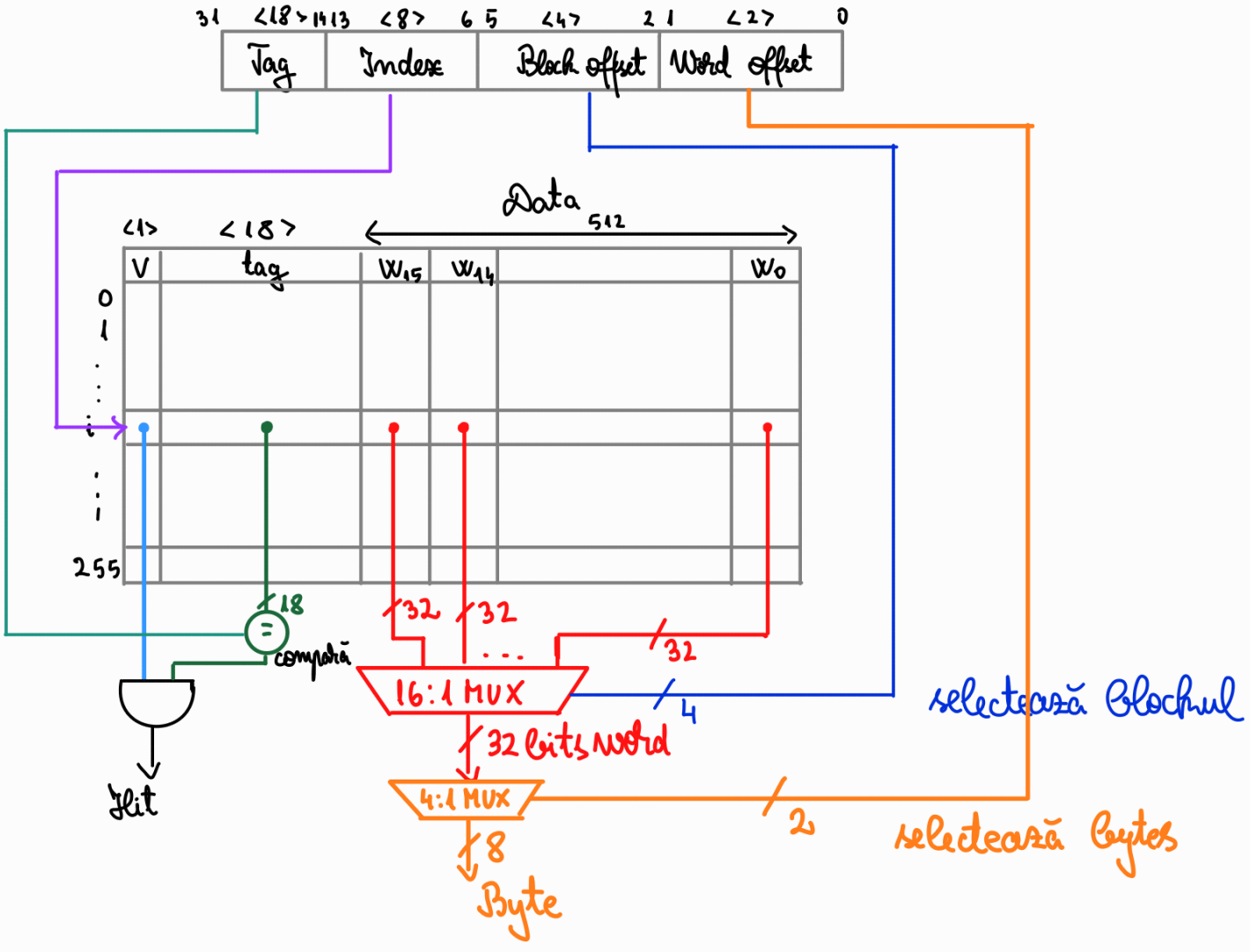
Intrinsity's Fast Math

Example ∘ MIPS architecture

- address word : 32 bits
- data word : 32 bits = 4 bytes (B) = 2^2 ⇒ word offset (W.O.) = 2
- split cache → data : 16 KiB
  → instructions: 16 KiB
- 1 block : 16 words = $2^4 \cdot 2^2 = 2^6$ B = $2^6 \cdot 2^3$ bits = 2^9 = 512 bits
  ⇒ block offset (B.O.) = 4
- byte addressing

1 Ki	$\longrightarrow 2^{10}$	1 K	$\rightarrow 10^3$
1 Mi	$\longrightarrow 2^{20}$	1 M	$\rightarrow 10^6$
1 Gi	$\longrightarrow 2^{30}$	1 G	$\rightarrow 10^9$
1 Ti	$\longrightarrow 2^{40}$	1 T	$\rightarrow 10^{12}$

- Cache size = 16 KiB = $2^4 \cdot 2^{10} = 2^{14}$ B = $\frac{2^{14}}{2^4 \cdot 2^2} = 2^8$ blocks = 256 blocks
  2^8 → index = 8 ⇒ tag = 32-8-6 = 18
- Main Memory = 4 GiB = $2^2 \cdot 2^{30}$ B = $\frac{2^{32}\text{ B}}{2^4 \cdot 2^2} = 2^{26}$ blocks
  2^{26} → m ⇒ tag = m - m = 26 - 8 = 18



Cache Performance Benchmarks

- Memory access → Hit / Miss
- Hit Rate = 1 - Miss Rate

3.1.2. Set-associative mapping

2^s → blocks at the same cache indese — k-way associative $k = 2^s$

$M_2 \rightarrow 2^m$ blocks ← j

$M_1 \rightarrow 2^n$ blocks $= 2^{n-s}$ sets ← i

$i = j \bmod 2^{n-s}$

$M_2(0), M_2(1), \ldots, M_2(j), \ldots, M_2(2^m-1)$

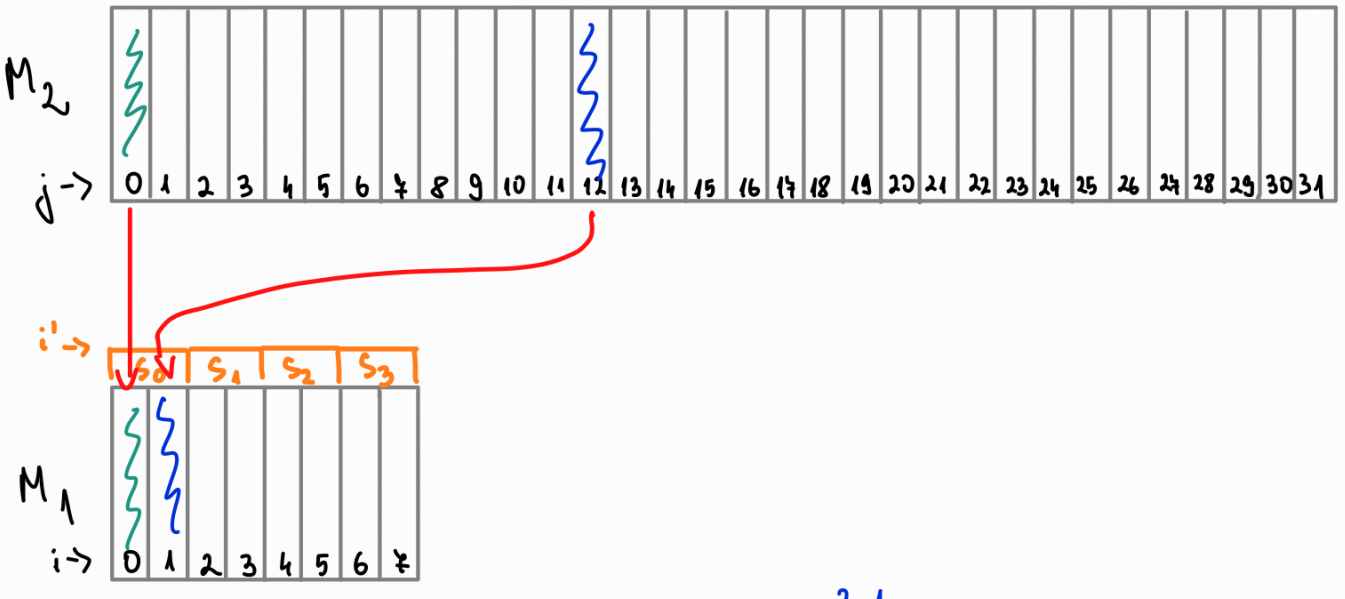
$M_1(0), M_1(1), \ldots, M_1(i), \ldots, M_1(2^n-1)$ $\quad m < n$

k-way associative mapping

$k = 2^s$

$M_1'(0), M_1'(1), \ldots M_1'(i'), \ldots, M_1'(2^{n-s}-1)$ $\quad i' = j \bmod 2^{n-s}$

exemplu: $m = 3$, $n = 5$, $s = 1$ ⇒ M_2 : 32 Blocuri



În cazul nostru $i' = j \bmod 2^{3-1} = j \bmod 4$

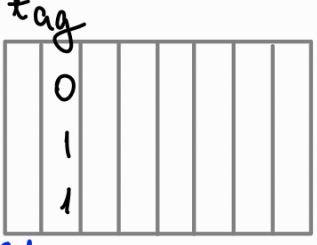
pt 12 de exe: $i' = 12 \bmod 4 = 0$

block indese: 0 1 1 0 0 înainte (m)

0 1 1 0 0 acum $(m-s)$ $= 2$

tag

indesetate 2 biți



Exemplul 2 : 24 de biți

MM : 2^{24} words — 2-WAY SA

1 word = 1 B

1 block = 4 words

size M_1 = 1 kBlock → 2^{10} blocks => index = $\frac{2^{10}}{2}$ = 9 (2-way)

Address	Code	
8192	230	2^{13}
4102	71	$2^{12}+2^{2}+2$
12292	140	$2^{13}+2^{12}+2^{2}$

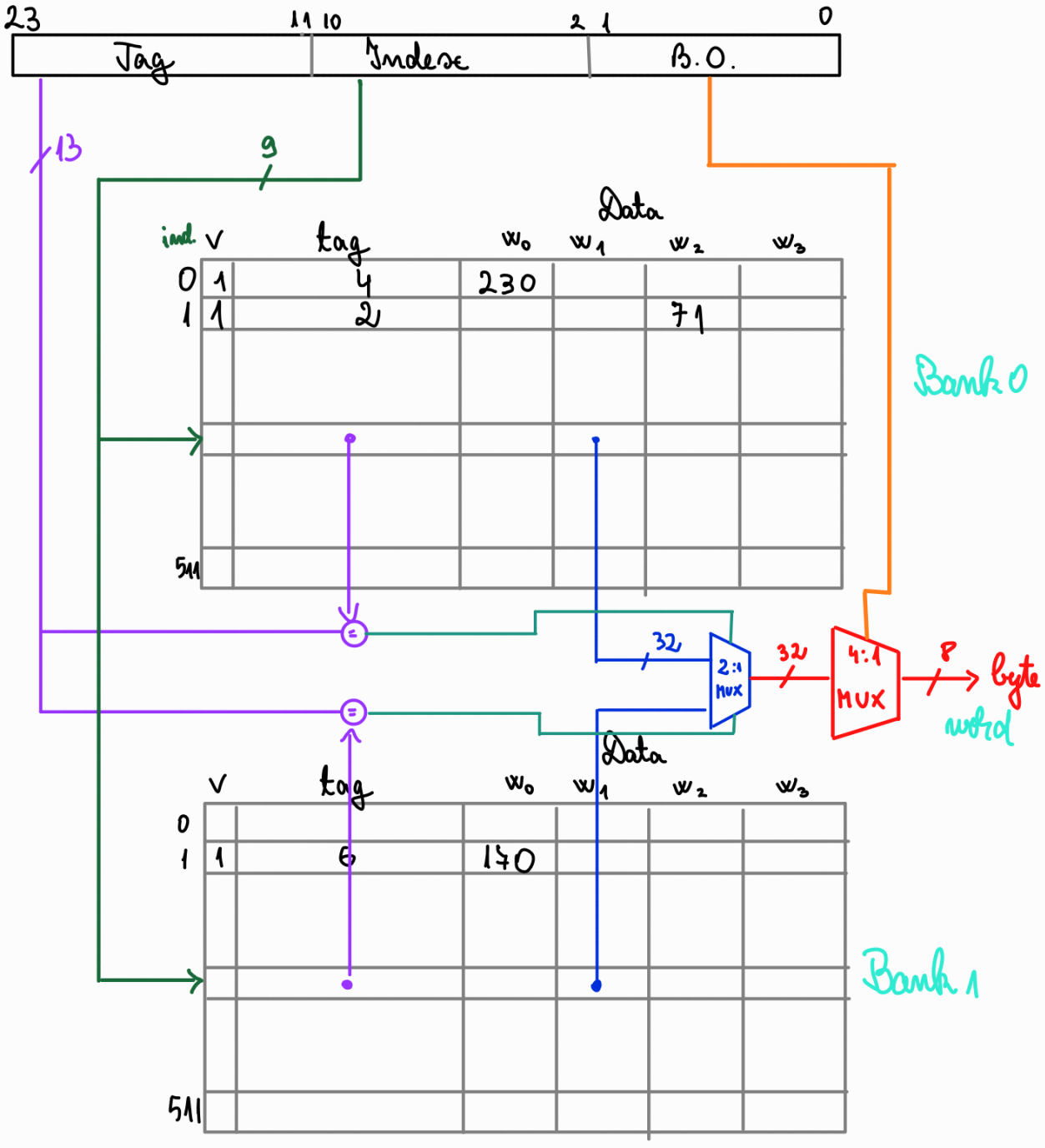
Address M_2 — Block index

0	0	1	2	3	0
4	4	5	6	7	1
8	8	9	10	11	2
⋮	⋮	⋮	⋮	⋮	⋮
4092	4092	4093	4094	4095	1023
4096	4096				1024
4100	4100	4101	4102 71	4103	1025
⋮	⋮	⋮	⋮	⋮	⋮
8188					2044
8192	8192 230	8193	8194	8195	2048
8196					2049
⋮	⋮	⋮	⋮	⋮	⋮
12288					3072
12292	12292 140	12293	12294	12295	3073
12296					3074
$2^{24}-4$					$2^{22}-1$

	Tag	Index	B.O.	W.O.
BM	<m-n>	<n>	<p>	<k>
2-WAY	<m-m+s>	<m-s>	<p>	<k>

	tag	index	BO
8192	00000000000100	000000000	00
4100	00000000000010	000000001	10
12292	00000000000110	000000001	00

merge în cache 2


23
11 10
2 1
0
Tag
Indexe
B.O.
13
9
Data
ind.
V
tag
w0
w1
w2
w3
0
1
511
1
4
230
2
71
Bank 0
=
=
32
2:1 MUX
32
4:1 MUX
8
byte word
Data
V
tag
w0
w1
w2
w3
0
1
511
1
6
170
Bank 1

Example 3: VAX 11/780

Size (MM) = 4 GiB = $2^2 \cdot 2^{30} = 2^{32}$ B ⇒ word address = 32

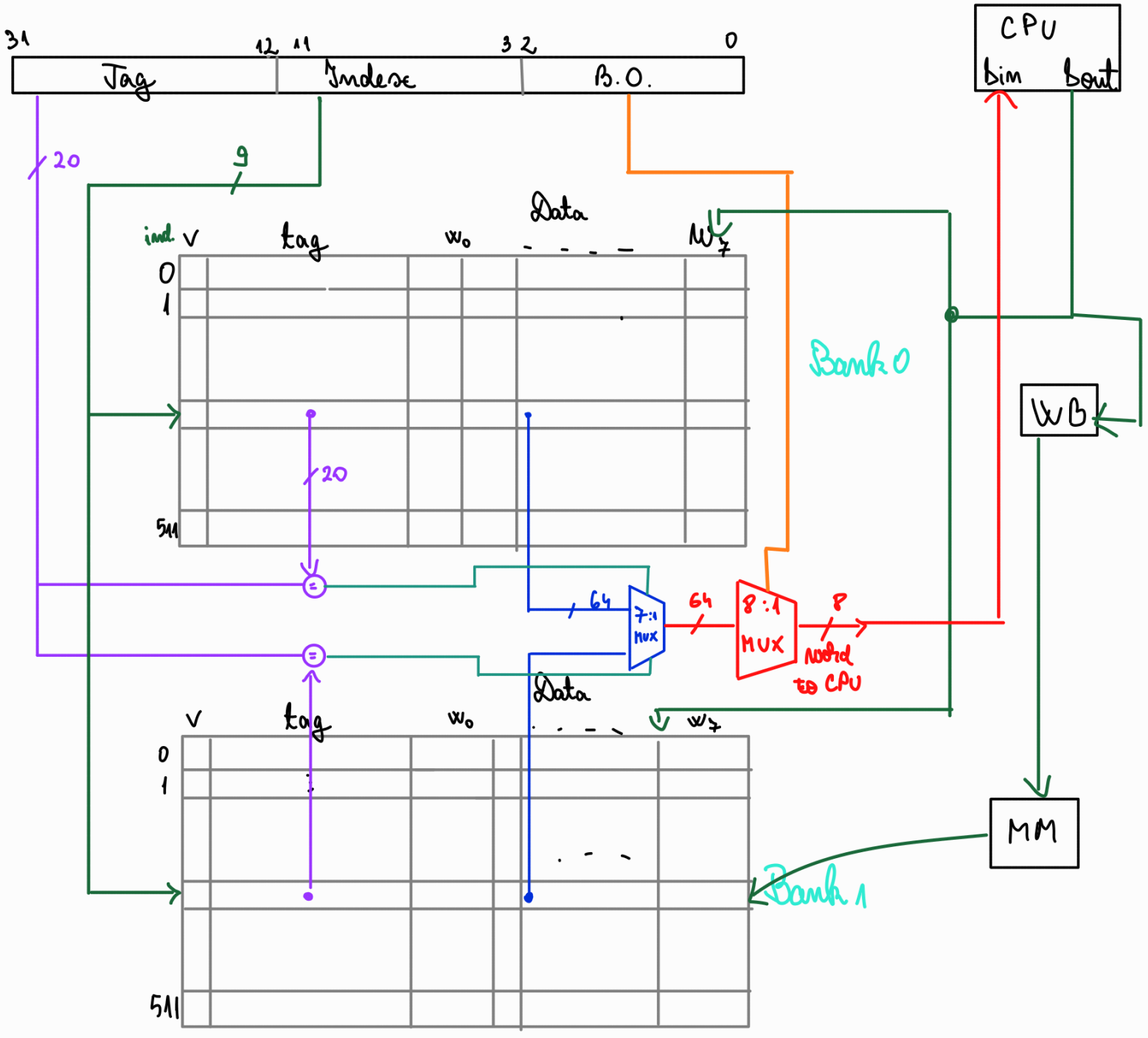
1 word = 1 B = 2^0 B ⇒ W.O. = 0

1 Block = 8 words = 2^3 words = 64 bits ⇒ B.O. = 3

Cache data size = 1 MBlock = $\frac{2^{10} \text{ Blocks}}{2^1}$ = 2^9 sets ⇒ index = 9

2-Way SA cache

tag = 32 - 3 - 9 = 20



Example 4 :

size MM = 4 GiB = $2^2 \cdot 2^{30} = 2^{32}$ B => Byte address = 32 bits

Byte addressable

1 word = 32 bits = $\frac{2^5}{2^3} = 2^2$ B => W.O. = 2

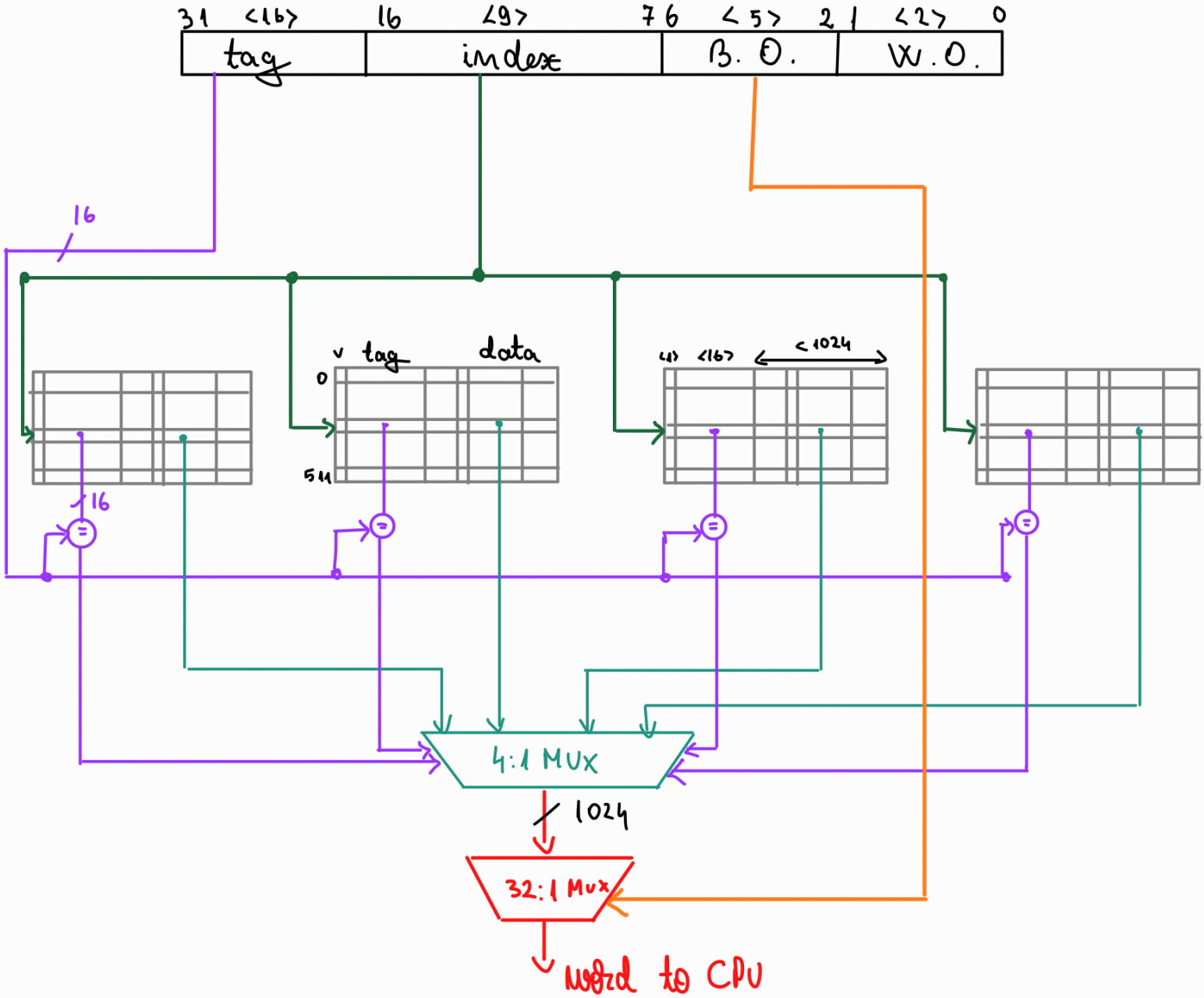
1 block = 32 words = 2^5 words => B.O. = 5

Data size (cache) = 256 KiB = $2^8 \cdot 2^{10}$ B = 2^{18} B

4-way SA cache => S=2 ($N = 2^2$)

1 block = 2^5 words = $2^5 \cdot 2^2 = 2^7$ => cache size = $\frac{2^{18}}{2^7} = 2^{11}$ blocks

$\frac{2^{11} \text{ blocks}}{2^2 \text{ blocks/set}} = 2^9$ sets => index = 9



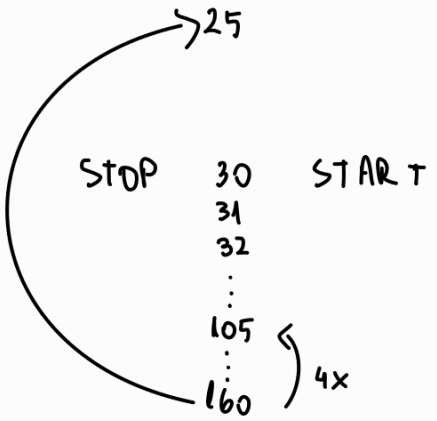
Example 5 — 4-WAY SA

- 8 blocks / bank ⇒ index = 3
- 4 words / block ⇒ B.O. = 2
- MM : 2^{16} words ⇒ $\frac{2^{16}}{2^2} = 2^{14}$ blocks

} ⇒ 16 - 3 - 2 = 11 (tag)

→ words addressing

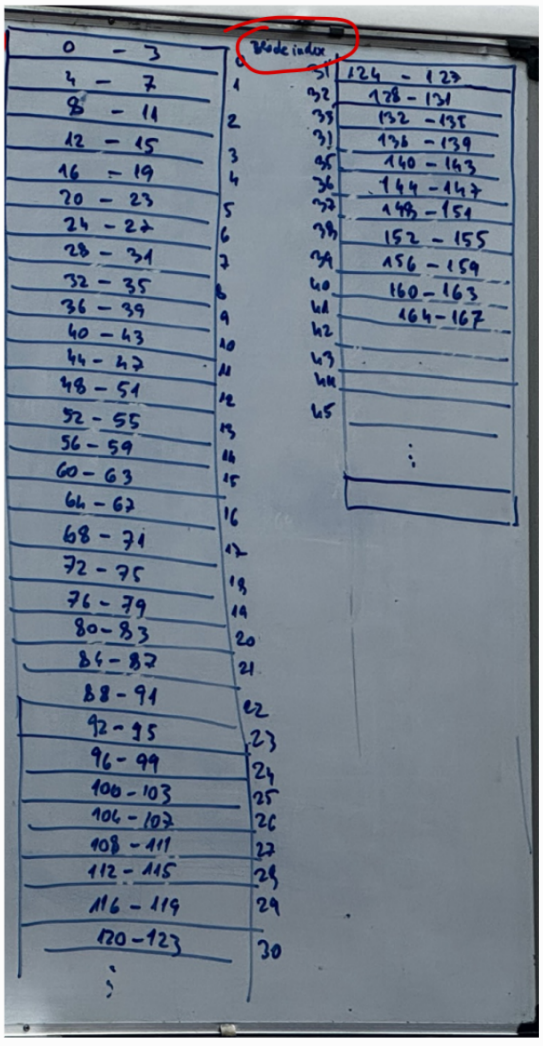
- Initially empty
- write back
- LRU replacement
- cache access time ⇒ $t_{ac} = 2$ ms
- miss penalty = $t_{mp} = 20$ ms



a) address format

b) hit ratio + average memory access time (AMAT)





30 = ...000 1 1 1 10 → 28-31 M (B.O.)

31 = ..000 111 11 → 28-31 H (e adus în cache)

32 = ...001 000 00 → 32-35 M

33, 34, 35 → 3H, 1M, 2H până la 103 incl 31 de ori

104 = ..0011 010 00

	✓	tag	Data
0	1	1	32-35
1			
2			
3			
4			
5			
6			
7	1	0	28-31

	✓	tag	Data
0			
1			
2			
3			
4			
5			
6			
7			

	✓	tag	Data
0			
1			
2			
3			
4			
5			
6			
7			

	✓	tag	Data
0			
1			
2			
3			
4			
5			
6			
7			

$$\begin{pmatrix} 1 \text{ miss} \\ 3 \text{ hit} \end{pmatrix} \cdot 31 \text{ ori} + \begin{pmatrix} 1 \text{ miss} \\ 1 \text{ hit} \end{pmatrix} +$$

inițial

După completarea cache-ului avem

325 hits

36 misses

361 access

$$AMAT = t_{ac} + MR \cdot \overset{\uparrow}{MP} = 2 + \frac{36}{361} \cdot 20 = 2 + 0{,}1 \cdot 20 = 4 \text{ ns}$$

Example 6 4-WAY SA cache

- size (MM) = 4 GiB = $2^2 \cdot 2^{30}$ B = 2^{32} B ⇒ word address = 32 bits
- byte address
- 1 word = 4 B = 2^2 B ⇒ W.O. = 2
- 1 block = 8 words = 2^3 words ⇒ B.O. = 3
- cache data size = 16 KiB = $2^4 \cdot 2^{10} = 2^{14}$

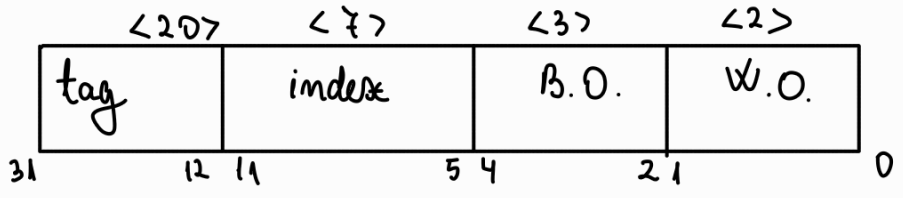
a) address format=?

1 block = 2^3 words = $2^3 \cdot 2^2$ B = 2^5 B

$$\text{cache size} = \frac{2^{14}\text{ B/cache}}{2^5\text{ B/block}} = 2^9 \text{ blocks/cache}$$

$$\text{bank size} = \frac{2^9}{2^2} = 2^7 \text{ blocks/bank} \Rightarrow \text{index} = 7$$

tag = 32 - 7 - 3 - 2 = 20



b) total cache size ?

$$\underbrace{2^2}_{\text{s.a.}} \cdot \underbrace{2^7}_{\text{lines}} \cdot \left(\underbrace{1\text{ b}}_{\text{valid bit}} + 20\text{ b} + 2^5 \cdot 2^3\text{ b} \right) = 2^9 \cdot 277\text{ b}$$

Extra

Full associative mapping

k - associativity : $k = 2^s$

$s = 0 \Rightarrow$ direct mapping

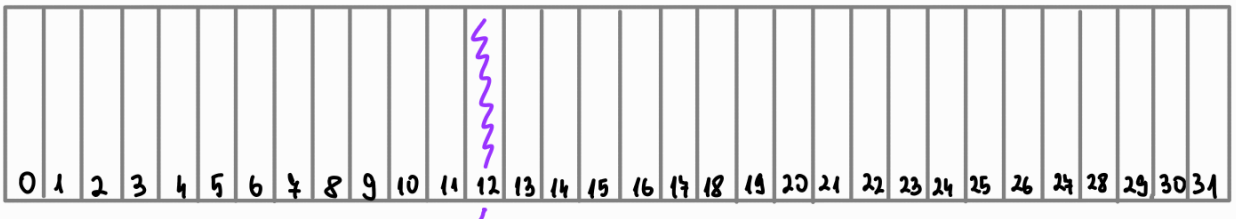
$s = m \Rightarrow$ full associativity unde 2^m = nr. of blocks in cache

Example 1

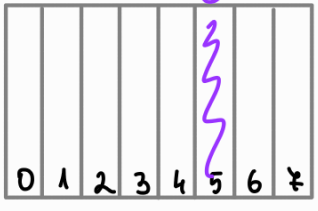
Size (M_2) = 2^n blocks , $n = 5$

Size (M_1) = 2^m blocks , $m = 3$

M_2



M_1



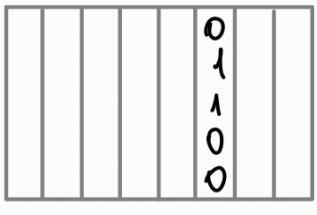
indexate de forma

8 blocuri ce nu mai sunt indexate ird, că sunt toate la același index

12 poate fi mapat oriunde: de ex pe 5

⟶ trb. scris tot nr, nu doar 0 parte din el

Tag



Example 2

2^{24} words

MM size = 16MiB = $2^4 \cdot 2^{20}$ B = 2^{24}

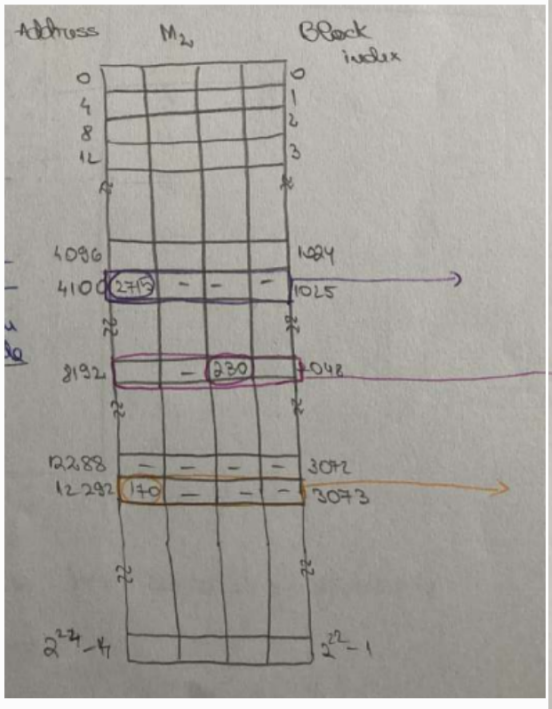
1 word = 1B = 2^0 $\Rightarrow$ W.O. = 0

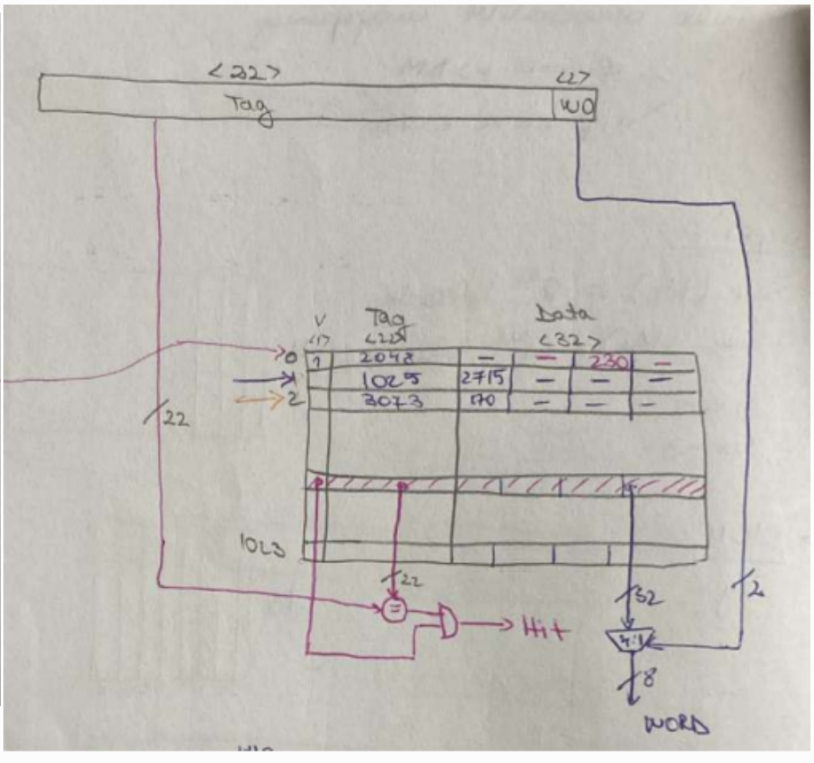
1 block = 4 words = 2^2 words $\Rightarrow$ B.O. = 2

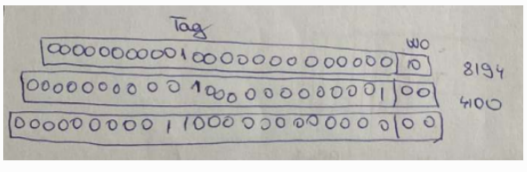
cache data size = 1 k blocks

$\Rightarrow$ tag = $\frac{2^{24}}{2^2} = 22$

address	code
8192	230
4100	2715
12 292	170







→ se pun la prima poz liberă din Data